1.1 PYTHON CODE FOR CONFIGURATION AND SETUP

This module handles the initial setup of the application, including environment variables, API keys, and configuration settings. It ensures that all dependencies and parameters required for smooth execution are properly defined and managed.

```
from pydantic_settings import BaseSettings
class Settings(BaseSettings):
"""Application settings loaded from environment variables."""

# Google Gemini Configuration
GOOGLE_API_KEY: str = "AIzaSyAQLHWUg7zc2mo7NTdAhr1J8GhVGsTSwqI"
GEMINI_MODEL: str = "models/gemini-2.5-flash"
GEMINI_EMBEDDING_MODEL: str = "models/gemini-embedding-001"

# Vector Store Configuration
VECTOR_STORE_TYPE: str = "chroma"  # chroma or faiss
CHROMA_PERSIST_DIR: str = "./data/chroma_db"
FAISS_INDEX_PATH: str = "./data/faiss_index"

# Document Processing
UPLOAD_DIR: str = "./data/uploads"
CHUNK_SIZE: int = 1000
CHUNK_OVERLAP: int = 200

# Retrieval Configuration
TOP_K_RESULTS: int = 5
SIMILARITY_THRESHOLD: float = 0.7

# Server Configuration
HOST: str = "0.0.0

# weather
OPENWEATHER_API_KEY: str | None = None

class Config:
```

```
        env_file = ".env"
        env_file_encoding = "utf-8"

settings = Settings()



# LLM and RAG
langchain>=0.1.16
langchain-core>=0.1.45
langchain-community>=0.0.32
langchain-google-genai>=0.0.11
google-generativeai>=0.4.1

# Vector Store
chromadb>=0.4.24
langchain-chroma>=0.1.0
faiss-cpu>=1.13.0

# Document Processing
pypdf>=3.17.4
python-docx>=1.1.0
docx2txt>=0.8
unstructured>=0.18.0
tiktoken>=0.7.0

# Web Framework
fastapi>=0.109.2
uvicorn>=0.27.1
python-multipart>=0.0.9

# Utilities
python-dotenv>=1.0.1
pydantic>=1.10.13
```

```
# Frontend
jinja2>=3.1.3
aiofiles>=23.2.1

# Core dependency fix for Python 3.14
numpy>=2.0.0



# Google Gemini Configuration
GOOGLE_API_KEY=AIzaSyAQLHWUg7zc2mo7NTdAhr1J8GhVGsTSwqI
GEMINI_MODEL=models/gemini-2.5-flash
GEMINI_EMBEDDING_MODEL=models/gemini-embedding-001

# Vector Store Configuration
VECTOR_STORE_TYPE=chroma
CHROMA_PERSIST_DIR=./data/chroma_db
FAISS_INDEX_PATH=./data/faiss_index

# Document Processing
UPLOAD_DIR=./data/uploads
CHUNK_SIZE=1000
CHUNK_OVERLAP=200

# Retrieval Configuration
TOP_K_RESULTS=5
SIMILARITY_THRESHOLD=0.7

# Server Configuration
HOST=0.0.0.0
PORT=8000
DEBUG=False

# real time weather
OPENWEATHER_API_KEY
```

1.2 PYTHON CODE FOR DOCUMENT PROCESSING AND EMBEDDING

This component is responsible for loading, cleaning, and transforming input documents into structured text. It then converts the processed data into vector embeddings, enabling efficient storage and semantic search capabilities.

```
"""Document loader and processor for various file types."""
import os
from typing import List
from langchain_core.documents import Document

from langchain_text_splitters import RecursiveCharacterTextSplitter

from langchain_community.document_loaders import (
    PyPDFLoader,
    Docx2txtLoader,
    TextLoader,
    UnstructuredHTMLLoader,
    CSVLoader,
)
from config import settings

class DocumentProcessor:
    """Handles loading and processing documents for RAG."""

    SUPPORTED_EXTENSIONS = {
        ".pdf": PyPDFLoader,
        ".docx": Docx2txtLoader,
        ".txt": TextLoader,
        ".html": UnstructuredHTMLLoader,
        ".csv": CSVLoader,
    }

    def __init__(
        self,
        chunk_size: int = None,
        chunk_overlap: int = None
    ):
```

```
        """Initialize the document processor.

        Args:
            chunk_size: Size of text chunks (default from settings)
            chunk_overlap: Overlap between chunks (default from settings)
        """
        self.chunk_size = chunk_size or settings.CHUNK_SIZE
        self.chunk_overlap = chunk_overlap or settings.CHUNK_OVERLAP
        self.text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=self.chunk_size,
            chunk_overlap=self.chunk_overlap,
            length_function=len,
            separators=["\n\n", "\n", " ", ""]
        )

    def load_document(self, file_path: str) -> List[Document]:
        """Load a single document from file path.

        Args:
            file_path: Path to the document file

        Returns:
            List of Document objects

        Raises:
            ValueError: If file type is not supported
        """
        ext = os.path.splitext(file_path)[1].lower()

        if ext not in self.SUPPORTED_EXTENSIONS:
            raise ValueError(
                f"Unsupported file type: {ext}. "
                f"Supported types: {list(self.SUPPORTED_EXTENSIONS.keys())}"
            )

        loader_class = self.SUPPORTED_EXTENSIONS[ext]
        loader = loader_class(file_path)
        documents = loader.load()

        # Add metadata
        for doc in documents:
            doc.metadata["source"] = file_path
            doc.metadata["file_type"] = ext

        return documents
```

```
def load_directory(self, directory_path: str) -> List[Document]:
    """Load all supported documents from a directory.

    Args:
        directory_path: Path to the directory

    Returns:
        List of Document objects from all files
    """
    documents = []

    for root, _, files in os.walk(directory_path):
        for file in files:
            ext = os.path.splitext(file)[1].lower()
            if ext in self.SUPPORTED_EXTENSIONS:
                file_path = os.path.join(root, file)
                try:
                    docs = self.load_document(file_path)
                    documents.extend(docs)
                except Exception as e:
                    print(f"Error loading {file_path}: {e}")

    return documents

def split_documents(self, documents: List[Document]) -> List[Document]:
    """Split documents into smaller chunks.

    Args:
        documents: List of Document objects to split

    Returns:
        List of chunked Document objects
    """
    return self.text_splitter.split_documents(documents)

def process_file(self, file_path: str) -> List[Document]:
    """Load and split a single file.

    Args:
        file_path: Path to the file

    Returns:
        List of chunked Document objects
    """
    documents = self.load_document(file_path)
    return self.split_documents(documents)
```

```
def process_directory(self, directory_path: str) -> List[Document]:
    """Load and split all files in a directory.

    Args:
        directory_path: Path to the directory

    Returns:
        List of chunked Document objects
    """
    documents = self.load_directory(directory_path)
    return self.split_documents(documents)
```

1.3 PYTHON CODE FOR RETRIEVAL SYSTEM

The retrieval system fetches relevant information from the vector database based on user queries. It uses similarity search techniques to identify the most meaningful documents for generating accurate responses.

```
"""Vector store management for document embeddings."""
import os
from typing import List, Optional
from langchain_core.documents import Document
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.vectorstores import FAISS
from config import settings


class VectorStoreManager:
    """Manages vector store operations for RAG."""


    def __init__(self, store_type: str = None):
        """Initialize the vector store manager.


        Args:
            store_type: Type of vector store ('chroma' or 'faiss')
        """
        self.store_type = store_type or settings.VECTOR_STORE_TYPE
        self.embeddings = self._get_embeddings()
        self.vector_store = None


    def _get_embeddings(self):
```

```
        """Get the appropriate embeddings model."""
        if not settings.GOOGLE_API_KEY:
            raise ValueError("GOOGLE_API_KEY must be set to generate embeddings with Gemini.")
        return GoogleGenerativeAIEmbeddings(
            model=settings.GEMINI_EMBEDDING_MODEL,
            google_api_key=settings.GOOGLE_API_KEY
        )

    def create_vector_store(self, documents: List[Document]) -> None:
        """Create a new vector store from documents.

        Args:
            documents: List of Document objects to index
        """
        if self.store_type == "chroma":
            # Ensure persist directory exists
            os.makedirs(settings.CHROMA_PERSIST_DIR, exist_ok=True)

            self.vector_store = Chroma.from_documents(
                documents=documents,
                embedding=self.embeddings,
                persist_directory=settings.CHROMA_PERSIST_DIR
            )
        elif self.store_type == "faiss":
            self.vector_store = FAISS.from_documents(
                documents=documents,
```

```
            embedding=self.embeddings
        )
        # Save FAISS index
        os.makedirs(os.path.dirname(settings.FAISS_INDEX_PATH), exist_ok=True)
        self.vector_store.save_local(settings.FAISS_INDEX_PATH)
    else:
        raise ValueError(f"Unsupported vector store type: {self.store_type}")

def load_vector_store(self) -> bool:
    """Load an existing vector store.

    Returns:
        True if loaded successfully, False otherwise
    """
    try:
        if self.store_type == "chroma":
            if os.path.exists(settings.CHROMA_PERSIST_DIR):
                self.vector_store = Chroma(
                    persist_directory=settings.CHROMA_PERSIST_DIR,
                    embedding_function=self.embeddings
                )
                return True
        elif self.store_type == "faiss":
            if os.path.exists(settings.FAISS_INDEX_PATH):
                self.vector_store = FAISS.load_local(
                    settings.FAISS_INDEX_PATH,
                    self.embeddings,
```

```
                allow_dangerous_deserialization=True
            )
            return True
        return False
    except Exception as e:
        print(f"Error loading vector store: {e}")
        return False

def add_documents(self, documents: List[Document]) -> None:
    """Add documents to an existing vector store.

    Args:
        documents: List of Document objects to add
    """
    if self.vector_store is None:
        self.create_vector_store(documents)
    else:
        self.vector_store.add_documents(documents)

        # Persist changes
        if self.store_type == "chroma":
            pass  # Chroma auto-persists
        elif self.store_type == "faiss":
            self.vector_store.save_local(settings.FAISS_INDEX_PATH)

def similarity_search(
    self,
```

```
    query: str,
    k: int = None,
    score_threshold: float = None
) -> List[Document]:
    """Search for similar documents.

    Args:
        query: Search query string
        k: Number of results to return
        score_threshold: Minimum similarity score

    Returns:
        List of relevant Document objects
    """
    if self.vector_store is None:
        if not self.load_vector_store():
            return []

    k = k or settings.TOP_K_RESULTS

    if score_threshold:
        results = self.vector_store.similarity_search_with_score(query, k=k)
        return [doc for doc, score in results if score >= score_threshold]

    return self.vector_store.similarity_search(query, k=k)

def similarity_search_with_scores(
```

```
    self,
    query: str,
    k: int = None
) -> List[tuple]:
    """Search for similar documents with relevance scores.

    Args:
        query: Search query string
        k: Number of results to return

    Returns:
        List of (Document, score) tuples
    """
    if self.vector_store is None:
        if not self.load_vector_store():
            return []

    k = k or settings.TOP_K_RESULTS
    return self.vector_store.similarity_search_with_score(query, k=k)

def delete_collection(self) -> None:
    """Delete the entire vector store collection."""
    if self.store_type == "chroma" and self.vector_store:
        self.vector_store.delete_collection()
    elif self.store_type == "faiss":
        if os.path.exists(settings.FAISS_INDEX_PATH):
            import shutil
```

```
            shutil.rmtree(settings.FAISS_INDEX_PATH)
        self.vector_store = None

    def get_retriever(self, k: int = None):
        """Get a retriever for use with LangChain.

        Args:
            k: Number of documents to retrieve

        Returns:
            A retriever object
        """
        if self.vector_store is None:
            self.load_vector_store()

        if self.vector_store is None:
            raise ValueError("No vector store available. Please add documents first.")

        k = k or settings.TOP_K_RESULTS
        return self.vector_store.as_retriever(search_kwargs={"k": k})
```

2 RESPONSE GENERATION & VALIDATION ENGINE

2.1 PYTHON CODE FOR CONSTRAINED LLM GENERATION

This module integrates with a Large Language Model (LLM) to generate responses based on user queries. It combines retrieved data with model intelligence to produce context-aware and meaningful outputs.

```
"""LLM client and RAG chain implementation."""
from typing import List, AsyncGenerator

from langchain_core.documents import Document
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

from langchain_google_genai import ChatGoogleGenerativeAI

from config import settings
from vector_store import VectorStoreManager

# Default RAG prompt template for Disaster Management
RAG_SYSTEM_PROMPT = """You are an expert Disaster Management AI Assistant
        designed to help emergency responders,
government officials, and citizens with disaster preparedness, response, and recovery
        information.

Use the following pieces of context to answer the user's question about disaster
        management.
If you don't know the answer based on the context, say that you don't know - DO NOT
        make up emergency procedures.

Context:
{context}
```

CRITICAL GUIDELINES:

1. SAFETY FIRST - Always prioritize life-saving information

2. Be CLEAR and ACTIONABLE - People may be in emergency situations

3. Provide STEP-BY-STEP instructions when explaining procedures

4. Include EMERGENCY CONTACT information when relevant (911, local emergency services)

5. Distinguish between IMMEDIATE actions and long-term recovery steps

6. If unsure about specific protocols, advise consulting local emergency management authorities

7. Consider vulnerable populations (elderly, disabled, children) in your recommendations

Disaster Categories You Can Help With:

- Natural Disasters: Earthquakes, Floods, Hurricanes, Tornadoes, Wildfires, Tsunamis

- Man-made Disasters: Industrial accidents, Chemical spills, Nuclear emergencies

- Public Health: Pandemics, Disease outbreaks

- Emergency Preparedness: Evacuation plans, Emergency kits, Shelter information

- Recovery: Post-disaster assistance, Insurance claims, Mental health support

IMPORTANT INSTRUCTION: You MUST translate your final answer and reply entirely in the following language: {language}.

"""

GENERAL_SYSTEM_PROMPT = """You are an expert Disaster Management AI Assistant designed to help emergency responders,

government officials, and citizens with disaster preparedness, response, and recovery information.

CRITICAL GUIDELINES:

1. SAFETY FIRST - Always prioritize life-saving information

2. Be CLEAR and ACTIONABLE - People may be in emergency situations

3. Provide STEP-BY-STEP instructions when explaining procedures

4. Include EMERGENCY CONTACT information when relevant (911, local emergency services)

5. Distinguish between IMMEDIATE actions and long-term recovery steps

6. If unsure about specific protocols, advise consulting local emergency management authorities

7. Consider vulnerable populations (elderly, disabled, children) in your recommendations

Disaster Categories You Can Help With:

- Natural Disasters: Earthquakes, Floods, Hurricanes, Tornadoes, Wildfires, Tsunamis

- Man-made Disasters: Industrial accidents, Chemical spills, Nuclear emergencies

- Public Health: Pandemics, Disease outbreaks

- Emergency Preparedness: Evacuation plans, Emergency kits, Shelter information

- Recovery: Post-disaster assistance, Insurance claims, Mental health support

IMPORTANT INSTRUCTION: You MUST translate your final answer and reply entirely in the following language: {language}.

"""

STANDALONE_QUESTION_PROMPT = """Given the following conversation history and a follow-up question,

rephrase the follow-up question to be a standalone question that captures all necessary context.

Chat History:

{chat_history}

Follow-up Question: {question}

Standalone Question:"""

```
class LLMClient:
    """Client for interacting with LLM models."""

    def __init__(self, model: str = None, temperature: float = 0.7):
        """Initialize the LLM client.

        Args:
            model: Model name to use (default from settings)
            temperature: Creativity parameter (0-1)
        """
        self.model = model or settings.GEMINI_MODEL
        self.temperature = temperature
        self.llm = self._initialize_llm()

    def _initialize_llm(self):
        """Initialize the LLM based on configuration."""
        if not settings.GOOGLE_API_KEY:
            raise ValueError("GOOGLE_API_KEY must be set to use the Gemini client.")
        return ChatGoogleGenerativeAI(
            model=self.model,
            google_api_key=settings.GOOGLE_API_KEY,
            temperature=self.temperature,
            convert_system_message_to_human=True
        )

    def chat(self, message: str, system_prompt: str = None) -> str:
        """Send a single message and get a response.

        Args:
            message: User message
```

```
        system_prompt: Optional system prompt

    Returns:
        AI response string
    """
    messages = []
    if system_prompt:
        messages.append(SystemMessage(content=system_prompt))
    messages.append(HumanMessage(content=message))

    response = self.llm.invoke(messages)
    return response.content

def chat_with_history(
    self,
    message: str,
    history: List[dict],
    system_prompt: str = None
) -> str:
    """Send a message with conversation history.

    Args:
        message: Current user message
        history: List of previous messages [{"role": "user/assistant", "content": "..."}]
        system_prompt: Optional system prompt

    Returns:
        AI response string
    """
    messages = []
```

```
    if system_prompt:
        messages.append(SystemMessage(content=system_prompt))

    for msg in history:
        if msg["role"] == "user":
            messages.append(HumanMessage(content=msg["content"]))
        else:
            messages.append(AIMessage(content=msg["content"]))

    messages.append(HumanMessage(content=message))

    response = self.llm.invoke(messages)
    return response.content

async def stream_chat(
    self,
    message: str,
    history: List[dict] = None,
    system_prompt: str = None
) -> AsyncGenerator[str, None]:
    """Stream a chat response.

    Args:
        message: User message
        history: Optional conversation history
        system_prompt: Optional system prompt

    Yields:
        Response chunks as they arrive
    """
```

```
        messages = []
        if system_prompt:
            messages.append(SystemMessage(content=system_prompt))

        if history:
            for msg in history:
                if msg["role"] == "user":
                    messages.append(HumanMessage(content=msg["content"]))
                else:
                    messages.append(AIMessage(content=msg["content"]))

        messages.append(HumanMessage(content=message))

        async for chunk in self.llm.astream(messages):
            if chunk.content:
                yield chunk.content

class RAGChain:
    """RAG chain for question answering with context retrieval."""

    def __init__(
        self,
        vector_store_manager: VectorStoreManager = None,
        llm_client: LLMClient = None,
        system_prompt: str = None
    ):
        """Initialize the RAG chain.

        Args:
            vector_store_manager: Vector store for retrieval
```

```
        llm_client: LLM client for generation
        system_prompt: Custom system prompt
    """
    self.vector_store = vector_store_manager or VectorStoreManager()
    self.llm_client = llm_client or LLMClient()
    self.system_prompt = system_prompt or RAG_SYSTEM_PROMPT

def _format_context(self, documents: List[Document]) -> str:
    """Format retrieved documents into context string."""
    context_parts = []
    for i, doc in enumerate(documents, 1):
        source = doc.metadata.get("source", "Unknown")
        context_parts.append(f"[Source {i}: {source}]\n{doc.page_content}")
    return "\n\n---\n\n".join(context_parts)

def query(
    self,
    question: str,
    chat_history: List[dict] = None,
    k: int = None,
    language: str = "English" # added language parameter

) -> dict:
    """Answer a question using RAG.

    Args:
        question: User question
        chat_history: Optional conversation history
        k: Number of documents to retrieve
```

```
    Returns:
        Dict with 'answer', 'sources', and 'context'
    """
    # Retrieve relevant documents
    documents = self.vector_store.similarity_search(question, k=k)

    if not documents:
        lang_prompt = GENERAL_SYSTEM_PROMPT.format(language=language)
        # No documents found — fall back to direct LLM response
        if chat_history:
            answer = self.llm_client.chat_with_history(
                question,
                chat_history,
                system_prompt=lang_prompt
            )
        else:
            answer = self.llm_client.chat(question, system_prompt=lang_prompt)
        return {
            "answer": answer,
            "sources": [],
            "context": ""
        }

    # Format context
    context = self._format_context(documents)

    # Build prompt with context
    prompt_with_context = self.system_prompt.format(context=context,
        language=language)
```

```
        # Get response
        if chat_history:
            answer = self.llm_client.chat_with_history(
                question,
                chat_history,
                system_prompt=prompt_with_context
            )
        else:
            answer = self.llm_client.chat(question, system_prompt=prompt_with_context)

        # Extract sources
        sources = [
            {
                "source": doc.metadata.get("source", "Unknown"),
                "content": doc.page_content[:200] + "..."
            }
            for doc in documents
        ]

        return {
            "answer": answer,
            "sources": sources,
            "context": context
        }

    async def stream_query(
        self,
        question: str,
        chat_history: List[dict] = None,
        k: int = None,
```

```
    language: str = "English" #added language parameter
) -> AsyncGenerator[dict, None]:
    """Stream a RAG response.

    Args:
        question: User question
        chat_history: Optional conversation history
        k: Number of documents to retrieve

    Yields:
        Dicts with 'type' ('chunk', 'sources', 'done') and relevant data
    """
    # Retrieve relevant documents
    documents = self.vector_store.similarity_search(question, k=k)

    if not documents:
        # formating the general prompt with the language parameter
        lang_prompt = GENERAL_SYSTEM_PROMPT.format(language=language)
        # No documents found — fall back to direct LLM streaming
        async for chunk in self.llm_client.stream_chat(
            question,
            history=chat_history,
            system_prompt=lang_prompt
        ):
            yield {"type": "chunk", "content": chunk}
        yield {"type": "sources", "sources": []}
        yield {"type": "done"}
        return

    # Format context
```

```
        context = self._format_context(documents)
        #build prompt with context and language
        prompt_with_context = self.system_prompt.format(context=context,
            language=language)

        # Stream response
        async for chunk in self.llm_client.stream_chat(
            question,
            history=chat_history,
            system_prompt=prompt_with_context
        ):
            yield {"type": "chunk", "content": chunk}

        # Send sources after streaming
        sources = [
            {
                "source": doc.metadata.get("source", "Unknown"),
                "content": doc.page_content[:200] + "..."
            }
            for doc in documents
        ]
        yield {"type": "sources", "sources": sources}
        yield {"type": "done"}

    def clear_memory(self):
        """Clear conversation memory."""
        pass
```

2.2 PYTHON CODE FOR VALIDATION AND GROUNDING

This component ensures that generated responses are accurate and grounded in retrieved data. It minimizes hallucinations by validating outputs against stored knowledge sources.

```
"""Vector store management for document embeddings."""
import os
from typing import List, Optional
from langchain_core.documents import Document
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.vectorstores import FAISS
from config import settings

class VectorStoreManager:
    """Manages vector store operations for RAG."""

    def __init__(self, store_type: str = None):
        """Initialize the vector store manager.

        Args:
            store_type: Type of vector store ('chroma' or 'faiss')
        """
        self.store_type = store_type or settings.VECTOR_STORE_TYPE
        self.embeddings = self._get_embeddings()
        self.vector_store = None

    def _get_embeddings(self):
        """Get the appropriate embeddings model."""
        if not settings.GOOGLE_API_KEY:
            raise ValueError("GOOGLE_API_KEY must be set to generate embeddings
            with Gemini.")
        return GoogleGenerativeAIEmbeddings(
            model=settings.GEMINI_EMBEDDING_MODEL,
            google_api_key=settings.GOOGLE_API_KEY
        )
```

```
def create_vector_store(self, documents: List[Document]) -> None:
    """Create a new vector store from documents.

    Args:
        documents: List of Document objects to index
    """
    if self.store_type == "chroma":
        # Ensure persist directory exists
        os.makedirs(settings.CHROMA_PERSIST_DIR, exist_ok=True)

        self.vector_store = Chroma.from_documents(
            documents=documents,
            embedding=self.embeddings,
            persist_directory=settings.CHROMA_PERSIST_DIR
        )
    elif self.store_type == "faiss":
        self.vector_store = FAISS.from_documents(
            documents=documents,
            embedding=self.embeddings
        )
        # Save FAISS index
        os.makedirs(os.path.dirname(settings.FAISS_INDEX_PATH),
        exist_ok=True)
        self.vector_store.save_local(settings.FAISS_INDEX_PATH)
    else:
        raise ValueError(f"Unsupported vector store type: {self.store_type}")

def load_vector_store(self) -> bool:
    """Load an existing vector store.

    Returns:
        True if loaded successfully, False otherwise
    """
    try:
        if self.store_type == "chroma":
            if os.path.exists(settings.CHROMA_PERSIST_DIR):
                self.vector_store = Chroma(
```

```
                persist_directory=settings.CHROMA_PERSIST_DIR,
                embedding_function=self.embeddings
            )
            return True
        elif self.store_type == "faiss":
            if os.path.exists(settings.FAISS_INDEX_PATH):
                self.vector_store = FAISS.load_local(
                    settings.FAISS_INDEX_PATH,
                    self.embeddings,
                    allow_dangerous_deserialization=True
                )
                return True
        return False
    except Exception as e:
        print(f"Error loading vector store: {e}")
        return False

def add_documents(self, documents: List[Document]) -> None:
    """Add documents to an existing vector store.

    Args:
        documents: List of Document objects to add
    """
    if self.vector_store is None:
        self.create_vector_store(documents)
    else:
        self.vector_store.add_documents(documents)

        # Persist changes
        if self.store_type == "chroma":
            pass  # Chroma auto-persists
        elif self.store_type == "faiss":
            self.vector_store.save_local(settings.FAISS_INDEX_PATH)

def similarity_search(
    self,
    query: str,
    k: int = None,
```

```
    score_threshold: float = None
) -> List[Document]:
    """Search for similar documents.

    Args:
        query: Search query string
        k: Number of results to return
        score_threshold: Minimum similarity score

    Returns:
        List of relevant Document objects
    """
    if self.vector_store is None:
        if not self.load_vector_store():
            return []

    k = k or settings.TOP_K_RESULTS

    if score_threshold:
        results = self.vector_store.similarity_search_with_score(query, k=k)
        return [doc for doc, score in results if score >= score_threshold]

    return self.vector_store.similarity_search(query, k=k)

def similarity_search_with_scores(
    self,
    query: str,
    k: int = None
) -> List[tuple]:
    """Search for similar documents with relevance scores.

    Args:
        query: Search query string
        k: Number of results to return

    Returns:
        List of (Document, score) tuples
    """
```

```
        if self.vector_store is None:
            if not self.load_vector_store():
                return []

        k = k or settings.TOP_K_RESULTS
        return self.vector_store.similarity_search_with_score(query, k=k)

    def delete_collection(self) -> None:
        """Delete the entire vector store collection."""
        if self.store_type == "chroma" and self.vector_store:
            self.vector_store.delete_collection()
        elif self.store_type == "faiss":
            if os.path.exists(settings.FAISS_INDEX_PATH):
                import shutil
                shutil.rmtree(settings.FAISS_INDEX_PATH)
        self.vector_store = None

    def get_retriever(self, k: int = None):
        """Get a retriever for use with LangChain.

        Args:
            k: Number of documents to retrieve

        Returns:
            A retriever object
        """
        if self.vector_store is None:
            self.load_vector_store()

        if self.vector_store is None:
            raise ValueError("No vector store available. Please add documents first.")

        k = k or settings.TOP_K_RESULTS
        return self.vector_store.as_retriever(search_kwargs={"k": k})
```

2.3 PYTHON CODE FOR SYSTEM EVALUATION AND METRICS

This module evaluates the system's performance by tracking response quality and efficiency. It helps in analyzing accuracy, relevance, and overall effectiveness of the chatbot system.

```
"""Pydantic models for API request/response schemas."""
from typing import List, Optional
from pydantic import BaseModel, Field

class ChatMessage(BaseModel):
    """A single chat message."""
    role: str = Field(..., description="Role of the message sender (user/assistant)")
    content: str = Field(..., description="Content of the message")

class ChatRequest(BaseModel):
    """Request body for chat endpoint."""
    message: str = Field(..., description="User's message/question")
    chat_history: Optional[List[ChatMessage]] = Field(
        default=None,
        description="Previous conversation history"
    )
    stream: bool = Field(default=False, description="Whether to stream the response")
    language: Optional[str] = Field(default="English", description="Target language for the
response")

class SourceInfo(BaseModel):
    """Information about a source document."""
    source: str = Field(..., description="File path or name of the source")
    content: str = Field(..., description="Excerpt from the source document")

class ChatResponse(BaseModel):
    """Response body for chat endpoint."""
    answer: str = Field(..., description="AI-generated answer")
    sources: List[SourceInfo] = Field(
        default=[],
        description="Source documents used for the answer"
    )

class DocumentUploadResponse(BaseModel):
    """Response after document upload."""
    message: str = Field(..., description="Status message")
    filename: str = Field(..., description="Name of uploaded file")
    chunks: int = Field(..., description="Number of chunks created")
```

```
class HealthResponse(BaseModel):
    """Health check response."""
    status: str = Field(..., description="Service status")
    vector_store: str = Field(..., description="Vector store status")
    documents_count: Optional[int] = Field(
        default=None,
        description="Number of documents in vector store"
    )

class ErrorResponse(BaseModel):
    """Error response body."""
    error: str = Field(..., description="Error message")
    detail: Optional[str] = Field(default=None, description="Detailed error info")

class DeleteResponse(BaseModel):
    """Response for delete operations."""
    message: str = Field(..., description="Status message")
    deleted: bool = Field(..., description="Whether deletion was successful"
```